# Thread Experiments Report

## 1. Atomicity

### Results from Multiple Runs:

To observe the race condition, the code was executed several times using go run atomic-counters.go.

**Outputs:**

```
molly@mollys-MacBook-Pro hw3 % go run atomic-counters.go
ops: 50000
opsRegular: 18357
molly@mollys-MacBook-Pro hw3 % go run atomic-counters.go
ops: 50000
opsRegular: 17350
molly@mollys-MacBook-Pro hw3 % go run atomic-counters.go
ops: 50000
opsRegular: 14507
molly@mollys-MacBook-Pro hw3 % go run atomic-counters.go
ops: 50000
opsRegular: 17751
```

- **Run 1:** ops: 50,000 | opsRegular: 18,357
- **Run 2:** ops: 50,000 | opsRegular: 17,350
- **Run 3:** ops: 50,000 | opsRegular: 14,507
- **Run 4:** ops: 50,000 | opsRegular: 17,751

### Analysis of Behavior:

- **Atomic counter (ops):** Consistently shows **50,000**. It utilizes atomic.Uint64 which leverages hardware-level CPU instructions (like Compare-And-Swap). This ensures that the increment is an "all-or-nothing" operation, preventing other threads from seeing an intermediate state.
- **Regular counter (opsRegular):** Shows significantly lower values. In Go, i++ is a non-atomic three-step process: (1) Read value from memory, (2) Increment value in CPU register, (3) Write value back to memory. When 50 goroutines do this simultaneously, multiple threads read the same old value before any can write the new one, causing "lost updates."

### With -race Flag:

Running the command `go run -race atomic-counters.go` triggered the following warning:

```
==================
WARNING: DATA RACE
Write at 0x00c000124038 by goroutine 9:
  main.main.func1()
      /Users/molly/CS6650/hw3/atomic-counters.go:19 +0x58
  sync.(*WaitGroup).Go.func1()
      /usr/local/go/src/sync/waitgroup.go:239 +0x54

Previous write at 0x00c000124038 by goroutine 14:
  main.main.func1()
      /Users/molly/CS6650/hw3/atomic-counters.go:19 +0x58
  sync.(*WaitGroup).Go.func1()
      /usr/local/go/src/sync/waitgroup.go:239 +0x54

Goroutine 9 (running) created at:
  sync.(*WaitGroup).Go()
      /usr/local/go/src/sync/waitgroup.go:237 +0x78
  main.main()
      /Users/molly/CS6650/hw3/atomic-counters.go:16 +0x8c

Goroutine 14 (running) created at:
  sync.(*WaitGroup).Go()
      /usr/local/go/src/sync/waitgroup.go:237 +0x78
  main.main()
      /Users/molly/CS6650/hw3/atomic-counters.go:16 +0x8c
==================
ops: 50000
opsRegular: 19688
Found 2 data race(s)
```

**Conclusion:** The race detector proves that unsynchronized concurrent access to opsRegular exists. Atomic operations guarantee safe concurrent access, whereas regular variables are susceptible to data corruption in multithreaded environments.

## 2. Shared Collections (Concurrent Maps)

**Observation:**

- **Plain Map:** Consistently crashed with fatal error: concurrent map writes.
- **Evidence:** As seen in the stack trace below, the Go runtime triggered a fatal error when multiple goroutines attempted to modify the map structure simultaneously.
- **The "Why":** Go's map implementation is highly optimized for speed but lacks internal locking to avoid the overhead of a Mutex on every access. In a distributed context, this represents the danger of "Shared State without Synchronization." The runtime chooses to panic rather than allow silent data corruption.

**Locking Tradeoff Analysis:**

```
--- Running Map Experiment: mutex ---
Mode: mutex | Length: 50000 | Elapsed Time: 45.110084ms

--- Running Map Experiment: rwmutex ---
Mode: rwmutex | Length: 50000 | Elapsed Time: 32.078375ms

--- Running Map Experiment: syncmap ---
Mode: syncmap | Length: 50000 | Elapsed Time: 35.006875ms
```

| Method | Execution Time | Structural Impact | Use Case Recommendation |
|---|---|---|---|
| **Standard Mutex** | **45.11ms** | Wraps map in a struct. | Best for simple, high-frequency write scenarios where absolute serialization is required. |
| **RWMutex** | **32.08ms** | Allows multiple readers, 1 writer. | Best for "Read-Heavy" systems (e.g., a cache). In our write-only test, the speed gain is likely due to Go's internal scheduling of write-lock acquisition. |
| **Sync.Map** | **35.01ms** | Uses atomic loading/storing. | Best for scenarios where keys are stable and there are many more reads than writes, or when different goroutines write to disjoint sets of keys. |

**Key Lesson:** In distributed systems, locking introduces "contention." While a Mutex ensures correctness, it effectively turns your parallel system into a serial one. Finding the right granularity of locking is essential for scalability.

## 3. File Access (The Cost of Persistence)

**Results:**

```
--- Running File Access Experiment ---
Unbuffered Write: 181.355834ms
Buffered Write:   16.4305ms
```

- **Unbuffered Write:** 181.36 ms
- **Buffered (bufio):** 16.43 ms (~11x faster)

**Deep Dive into Tradeoffs:**

The massive performance gap is due to the cost of **System Calls (Syscalls)**.

- **Unbuffered:** Each loop iteration triggers an OS syscall to move data from User Space to Kernel Space. This involves saving CPU registers, switching page tables, and crossing the kernel boundary—an extremely expensive operation.
- **Buffered:** bufio maintains a 4KB buffer in memory. It aggregates 100,000 small writes into a few large ones. This "Batching" strategy is a cornerstone of distributed system design: **Amortize the fixed cost of a high-latency operation (like Disk IO or Network**

**Calls) by sending more data at once.**

# 4. Context Switching (Lightweight vs. Heavyweight)

**Results:**

```
--- Context Switching ---
GOMAXPROCS(1): Total Duration: 465.111667ms | Avg Switch Cost: 232ns
GOMAXPROCS(8): Total Duration: 794.041458ms | Avg Switch Cost: 397ns
```

- **Single OS Thread (GOMAXPROCS 1):** 232 ns (Avg Switch Cost)
- **Multi OS Thread (GOMAXPROCS 8):** 397 ns (Avg Switch Cost)

**Analysis & Distributed Foundations:**

- **Why GOMAXPROCS(1) is faster:** When Go runs on a single thread, the "Context Switch" happens entirely in User Space. The Go scheduler simply swaps a few pointers to different goroutine stacks. The CPU cache remains "hot" because the execution stays on the same core.
- **Why Multi-thread is slower:** When switching across 8 threads, the OS kernel gets involved. This involves "Cache Invalidation" (moving data between L1/L2 caches of different cores) and potentially NUMA memory overhead.

**Connection to VMs and Containers:**

This experiment demonstrates why modern systems favor "Lightweight Threads" (Goroutines/Green Threads) over OS processes or Virtual Machines for high-concurrency tasks.

- **Process/VM Switch:** Involves full hardware state saves, flushing TLBs, and massive memory overhead (milliseconds).
- **Goroutine Switch:** Only saves minimal state (nanoseconds).
  In a "party of nodes" (distributed system), the cost of switching determines how many simultaneous connections a single node can handle before the "overhead" exceeds the "useful work."